

Planet Pulse

See what Earth is signaling now.

A Global Environmental Situation-Awareness Platform
Complete Project Architecture & Logic Document

Author: John Zhou

Date: July 2026

License: Apache License 2.0

GitHub: github.com/jzjzzzzzz/PlanetPulse

Live Demo: planet-pulse-eta.vercel.app

Table of Contents

1. Project Overview & Product Positioning
2. Technical Architecture
3. Data Flow & API Design
4. Core Algorithm: Hotspot Scoring System
5. Core Algorithm: Personal Relevance Scoring
6. Core Algorithm: Haversine Distance Calculation
7. Component Tree & UI Architecture
8. 3D Globe Rendering Engine
9. Data Sources & Resilience Mechanisms
10. Deployment & Operations
11. Testing System
12. Privacy & Data Ethics
13. Future Roadmap

1 Project Overview & Product Positioning

1.1 What is Planet Pulse

Planet Pulse is a global environmental situation-awareness website. It transforms scattered environmental data into a clear global overview and a personalized local environmental signal. The platform combines a cinematic interactive 3D Earth, real NASA environmental-event data, an explainable hotspot-ranking system, and location-aware personal relevance information.

1.2 Product Positioning

- Type: Environmental situation-awareness website (NOT an emergency-warning service)
- Purpose: Global environmental monitoring dashboard + Earth-systems education tool + NASA open-data consumer-grade interface
- Explicitly NOT: An emergency alert system, an official NASA product, or a replacement for local emergency services

1.3 Core Features

Feature	Description
3D Globe	Interactive dark-themed globe with atmospheric glow, color-coded event markers, animated pulse rings
Hotspot Ranking	Deterministic 0-100 scoring system with 6 weighted factors, human-readable score explanations
Personal Relevance	Four-dimensional weighted scoring: proximity x severity x recency x time-of-day context
Real Data Sources	NASA EONET v3 (environmental events) + NASA FIRMS (satellite fire detections, optional)
Responsive Design	Full desktop layout + simplified mobile layout, supports 375px phones to large displays

1.4 Technology Stack

Layer	Technology
Framework	Next.js 16 (App Router)
Language	TypeScript (strict mode)
Styling	Tailwind CSS v4
3D Engine	react-globe.gl + Three.js
Icons	Lucide React
Validation	Zod
Testing	Vitest
Deployment	Vercel
Package Manager	npm

2 Technical Architecture

2.1 High-Level Architecture

Planet Pulse follows a Next.js App Router architecture with strict client-server separation:

Client (Browser):

- 3D Globe (dynamic import, SSR disabled)
- Layer Panel (event category filtering)
- Hotspot Panel (global hotspot ranking, Top 8)
- Local Signal Bar (personal relevance signal)
- Event Details (event detail modal)
- Observation Timeline (observation history)
- Mobile Event Sheet (mobile bottom drawer)

Server (Next.js API Routes):

- /api/events --> NASA EONET v3 API
- /api/fires --> NASA FIRMS API
- /api/location --> Vercel IP Geolocation
- Scoring Engine (pure functions, independently testable)

2.2 Key Architectural Decisions

Decision	Rationale
Globe Component	Dynamic import + ssr:false; Three.js/WebGL is client-only
Data Fetching	All data goes through server-side API Routes; API keys never exposed to client
Scoring Logic	Isolated pure functions, zero side effects, 100% unit-testable
Stateless Design	No database, no auth, no CMS — fully stateless
Resilience	Three-tier degradation: Live -> Stale Cache -> Fallback Sample Data

2.3 Project File Structure

```
day03/
  app/
    layout.tsx      # Root layout (fonts + metadata)
    page.tsx        # Main page (state management + component orchestration)
    globals.css      # Global styles (CSS variables + dark theme)
    api/
      events/route.ts # Events API (EONET)
      fires/route.ts  # Fire API (FIRMS)
      location/route.ts # Location API (IP geolocation)
      health/route.ts  # Health check
      satellite/route.ts # Satellite imagery proxy
    components/
      globe/          # 3D globe components
      layout/         # Layout components (panels + status bar)
      events/         # Event display components
      ui/             # Generic UI components
    lib/
      scoring/        # Scoring algorithms (core logic)
      nasa/           # NASA data fetching + normalization
      geo/            # Geographic computation
      globe/          # Globe visualization utilities
```

location/	# Location resolution
formatting/	# Date/time formatting
types/	
environment.ts	# Core type definitions
tests/	# Unit tests

3 Data Flow & API Design

3.1 Data Acquisition Pipeline

The data flow follows a Server-to-Client unidirectional pattern to protect API keys:

1. User visits page -> Browser loads React application
2. Client calls `fetch("/api/events")` -> Next.js API Route
3. API Route calls `fetch(EONET_API_URL)` -> NASA EONET v3
4. EONET returns raw JSON -> Zod Schema validation
5. `normalizeEvent()` transforms data -> `EnvironmentalEvent[]`
6. Each event is scored via `computeHotspotScore()`
7. Results cached in memory (LiveCache, 30-min TTL)
8. Returned to client -> 3D globe + panels rendered
9. Client auto-refreshes every 60 seconds

3.2 API Endpoints

Endpoint	Purpose	Notes
GET /api/events	Environmental event list	Cache: s-maxage=600, swr=300
GET /api/fires	NASA FIRMS fire detections	Requires NASA_FIRMS_MAP_KEY
GET /api/location	User IP geolocation	Source: Vercel Headers
GET /api/health	Health check	Returns <code>{"status":"ok"}</code>

3.3 Three-Tier Resilience Strategy

The system implements a three-tier data degradation strategy to ensure normal operation under any network condition:

Tier 1 - Live Data (Normal mode)

- Real-time fetch from NASA EONET v3
- Updates in-memory cache (LiveCache) on success
- Cache TTL: 30 minutes

Tier 2 - Stale Cache (First line of defense when network fails)

- Returns the most recent successful cache
- Frontend marks data source as "stale-cache"

Tier 3 - Bundled Fallback (Last resort when no cache exists)

- Uses 8 sample events bundled at build time
- Covers all 9 event categories
- Frontend clearly labels data as sample data

3.4 Data Normalization Pipeline

The EONET v3 API supports both JSON and GeoJSON formats. The normalization pipeline handles:

1. Zod Schema Validation -> ensures data structure integrity

2. Category Mapping -> EONET category strings -> internal EventCategory enum
3. Coordinate Extraction -> Point: direct value; Polygon: centroid via Shoelace formula
4. Observation Extraction -> from geometry array, time-sorted + deduplicated (max 30)
5. Temporal Derivation -> startedAt (earliest observation) and updatedAt (latest observation)
6. Score Computation -> computeHotspotScore() executed for each event
7. Source Resolution -> extract data source name and URL

4 Core Algorithm: Hotspot Scoring System

4.1 Algorithm Overview

The Hotspot Score is a fully deterministic 0-100 scoring system. Given identical inputs and reference date, every invocation produces identical output. The algorithm uses no `Math.random()` or `Date.now()`, ensuring reproducibility across SSR, testing, and replay scenarios.

4.2 Scoring Factors & Weights

Factor	Range	Scoring Rule
1. Category Base	0-30	Wildfire/Volcano(25), Storm(20), Flood(18), Dust/Haze/Drought/Landslide(15), Ice(10), Other(10)
2. Data Recency	0-25	<6h:25, <12h:22, <24h:18, <48h:12, <7d:5, >7d:2, unknown:0
3. Magnitude Data	0-15	Value+Unit present:15, only one:8, neither:0
4. Event Status	0-10	Open/Active:10, Closed:0
5. Observations	0-10	>=5:10, >=3:7, >=2:5, 1:3
6. FIRMS Density	0-10	>100:10, >50:8, >20:6, >10:4, >0:2, 0:0

4.3 Algorithm Pseudocode

```
function computeHotspotScore(event, referenceDate):
  total = 0
  explanations = []

  // 1. Category weight
  total += CATEGORY_WEIGHTS[event.category] // 0-30
  explanations.push(categoryLabel)

  // 2. Recency
  hoursAgo = hoursBetween(referenceDate, event.updatedAt)
  recencyScore = bucketByRecency(hoursAgo) // 0-25
  total += recencyScore

  // 3. Magnitude
  if event.magnitudeValue AND event.magnitudeUnit:
    total += 15
  elif event.magnitudeValue OR event.magnitudeUnit:
    total += 8
  else: total += 0

  // 4. Status
  total += event.isOpen ? 10 : 0

  // 5. Observation count
  total += bucketByObsCount(event.observationCount) // 3-10

  // 6. FIRMS fire density
  total += bucketByFireDensity(event.nearbyFireCount) // 0-10

  return { score: min(total, 100), explanation }
```

4.4 Design Principles

- Fully deterministic: identical inputs always yield identical outputs
- Explainable: every score comes with a human-readable explanation list

- Pure function: no external state dependencies, no side effects
- Independently testable: fully testable without mocks

5 Core Algorithm: Personal Relevance Scoring

5.1 Algorithm Overview

The Personal Relevance Score quantifies how important an environmental event is to a specific user. It computes a weighted blend across four dimensions, with results in the range 0-100.

5.2 Scoring Factors

Dimension	Weight	Calculation
1. Proximity	45%	<50km:45, <100km:35, <250km:25, <500km:15, <1000km:5, >1000km:0
2. Hotspot Severity	30%	0.3 x event Hotspot Score
3. Data Recency	15%	<6h:15, <12h:12, <24h:9, <48h:5, <72h:2, older:0
4. Time Context	10%	Daytime (06-20) + <24h data:10, daytime only:5, nighttime/unknown:0

5.3 Scoring Formula

```
PersonalRelevance = Proximity + Severity + Recency + TimeContext

Proximity    = f(distanceKm)          x 0.45  (inverse-distance buckets)
Severity     = 0.3 x hotspotScore      (severity scaling)
Recency      = f(recencyHours)         x 0.15  (recency buckets)
TimeContext  = f(isDaytime, recency) x 0.10  (daytime + freshness)

Result = clamp(round(raw), 0, 100)
```

5.4 Application

- Local Signal Bar: bottom bar displaying nearest threat event and personal match percentage
- Nearest Event: auto-locates the globe to the closest environmental event to the user
- Color coding: >70 green (high attention), >40 yellow (moderate), <40 gray (low attention)

6 Core Algorithm: Haversine Distance Calculation

6.1 Algorithm Overview

The Haversine formula computes the great-circle distance between two points on the Earth's surface. It assumes a spherical Earth (radius 6371 km), which is sufficiently accurate for environmental monitoring scenarios.

6.2 Mathematical Formula

Haversine Formula:

```
a = sin^2(DeltaPhi/2) + cos(Phi1) * cos(Phi2) * sin^2(DeltaLambda/2)
c = 2 * atan2(sqrt(a), sqrt(1-a))
d = R * c
```

where:

```
Phi = latitude in radians
Lambda = longitude in radians
R = 6,371 km (Earth mean radius)
d = great-circle distance in km
```

6.3 Implementation Features

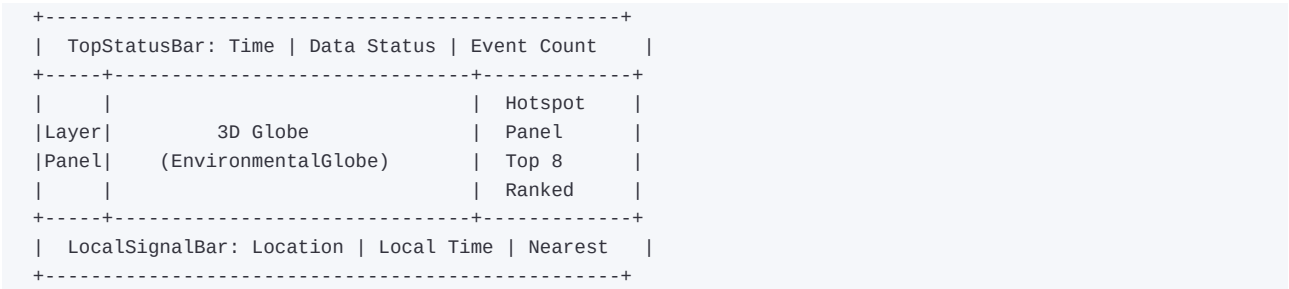
- Pure function, zero external dependencies
- Supports Node.js / Edge / Browser runtimes
- Input validation: coordinate range checking [-90,90] x [-180,180]
- Short-circuit optimization: identical coordinates return 0, avoiding floating-point issues
- Complete JSDoc documentation with mathematical derivation

6.4 Accuracy Notes

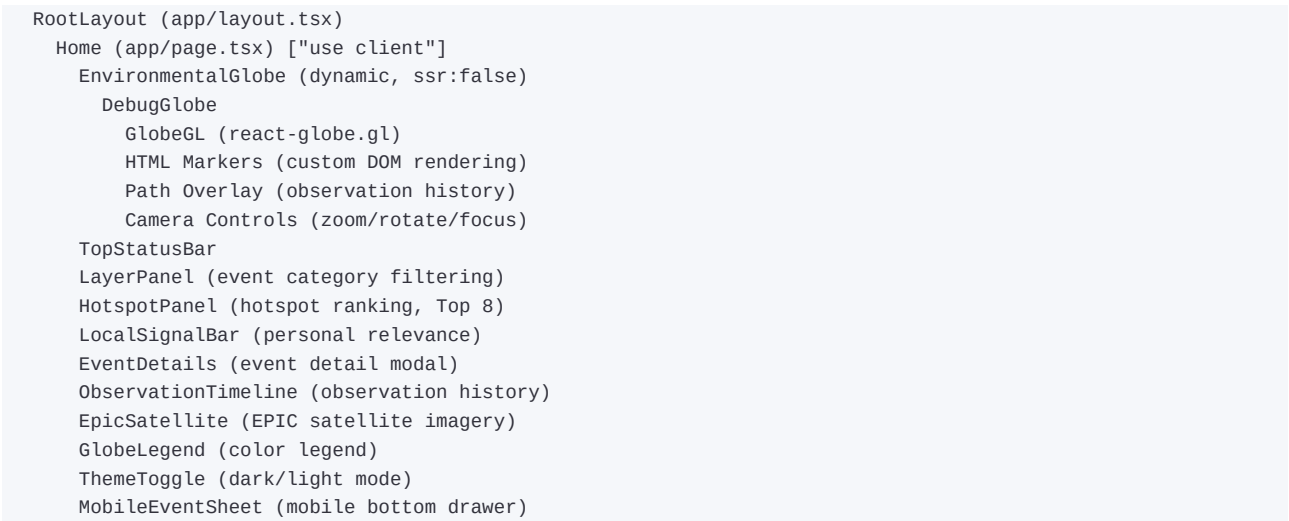
The Haversine formula has approximately 0.3% error under the spherical Earth approximation, which is more than sufficient for global environmental event positioning (typically at 10+ km scale). For higher precision (centimeter-level), the Vincenty formula can be substituted. The current use case does not require this precision.

7 Component Tree & UI Architecture

7.1 Page Layout (Desktop)



7.2 Complete Component Tree



7.3 State Management (Data Flow)

The Home component centrally manages all state and passes it to child components via props:

Data State:

events[], eventsLoading, eventsError, dataSource, lastUpdated

Location State:

userLocation { city, country, lat, lng, timezone }, hasFirmsKey

UI State:

selectedEventId, selectedCategory, refreshIn, showInfo

Derived Data (useMemo):

filteredEvents, nearestEvent, nearestDistance, personalRelevance,
localTime, categoryCounts, selectedEvent

8 3D Globe Rendering Engine

8.1 Technology Selection

The globe uses react-globe.gl (v2.38) wrapping Three.js for 3D Earth rendering. react-globe.gl is the most mature React 3D globe library, providing rich visualization capabilities for markers, paths, and atmosphere.

Key Decision: Using the htmlElementsData API (HTML markers) rather than native Three.js markers. The HTML marker system is more reliable and supports complex styling and event handling.

8.2 Rendering Configuration

Property	Value	Description
globeImageUrl	/textures/earth-light.jpg	2K Earth texture (jsDelivr CDN)
backgroundColor	Dark #0a1226 / Light #d5dbe3	Follows system theme
atmosphereColor	#5599DD / #3388CC	Atmospheric glow effect
atmosphereAltitude	0.3	Atmosphere layer height
htmlElementsData	MarkerData[]	HTML marker data source

8.3 Marker System

- Density Control: dynamically filters low-score events based on zoom level (alt>4: minScore 50, alt>3: minScore 30)
- Maximum 500 markers displayed, sorted by hotspotScore descending
- User Location: blue diamond rotated 45 degrees, visually distinct from other markers
- Selected Marker: white border + scale(1.5) enlargement effect
- Event Markers: circular, size = 6 + (score/100)*8 px
- Colors: 9 category colors from CATEGORY_COLORS_HEX palette

8.4 Camera Controls

- Zoom In/Out: altitude range [0.8, 8.0], multiplicative scaling
- Auto-Rotate: starts after 3 seconds of inactivity, pauses on user interaction
- Focus Event: camera flies to selected event coordinates (800ms animation)
- Observation Jump: camera flies to specific observation coordinate on click

8.5 Visual Effects

- Observation History Path: dashed path showing observation position changes for selected event (max 30 points)
- Path Animation: dashAnimateTime=8000ms flowing dashed-line effect
- Dark/Light Theme: globe background and atmosphere color auto-switch with system theme

9 Data Sources & Resilience Mechanisms

9.1 NASA EONET v3

The NASA Earth Observatory Natural Event Tracker (EONET) v3 is the primary data source.

API Endpoint: <https://eonet.gsfc.nasa.gov/api/v3/events>
Request Parameters: status=open&days=60&limit=500
Data Format: JSON (preferred) / GeoJSON (fallback)
Timeout: 15 seconds
Retry Strategy: up to 2 attempts, exponential backoff (500ms, 1500ms)

9.2 Event Category Mapping

EONET Category	Internal Enum	Description
Wildfires	wildfire	Active fire events
Severe Storms	severe-storm	Tropical cyclones, hurricanes, severe weather
Volcanoes	volcano	Volcanic activity and eruptions
Floods	flood	Flooding and inundation events
Drought	drought	Drought conditions and water scarcity
Dust and Haze	dust-haze	Dust storms, haze, and air quality events
Landslides	landslide	Landslides and mudslides
Sea and Lake Ice	sea-lake-ice	Ice extent, icebergs, and ice shelf events

9.3 Keyword Fallback Matching

When EONET category mapping fails, the event title is matched against keyword patterns using regular expressions.

Supported	keywords	include:
wildfire/fire/storm/cyclone/hurricane/volcano/flood/drought/dust/haze/landslide/sea ice/lake ice, and related variants.		

9.4 Geometry Data Processing

Point type: Direct coordinate extraction [lng, lat]
Polygon type: Centroid calculation via coordinate arithmetic mean or Shoelace formula
Other types (LineString, etc.): Skipped
Selection rule: Latest-dated valid geometry record

9.5 NASA FIRMS (Optional)

NASA FIRMS (Fire Information for Resource Management System) provides satellite thermal anomaly detections.

API: firms.modaps.eosdis.nasa.gov/api/area/
Configuration: Requires NASA_FIRMS_MAP_KEY (free registration)
When unconfigured: Frontend gracefully degrades, displaying "Detailed fire detections unavailable"

10 Deployment & Operations

10.1 Deployment Platform

Item	Value
Production URL	planet-pulse-eta.vercel.app
Platform	Vercel (Hobby Plan)
Framework	Next.js 16 App Router
Build Command	next build
Branch	main (auto-deploy)
Env Variables	NEXT_PUBLIC_SITE_URL, NASA_FIRMS_MAP_KEY

10.2 Verified Routes

Route	Status	Description
/	200 OK	Full Planet Pulse application
/api/health	200 OK	{"status":"ok"}
/api/events	200 OK	EONET live data + degradation
/api/location	200 OK	Vercel IP geolocation
/api/fires	200 OK	Graceful degradation (no FIRMS key)

10.3 Caching Strategy

- API Cache: Cache-Control: public, s-maxage=600, stale-while-revalidate=300
- Server Memory Cache: 30-minute TTL (LiveCache)
- Client Auto-Refresh: polls every 60 seconds

10.4 Observability

- Response Header: X-Data-Source (live / stale-cache / fallback)
- Degradation Reason: X-Degraded-Reason header with specific failure cause
- Logging: server-side console.error for API exceptions

11 Testing System

11.1 Testing Framework

- Framework: Vitest (deeply integrated with Vite ecosystem)
- Run: npm test (vitest run)
- Type Check: npm run typecheck (tsc --noEmit)
- Lint: npm run lint (ESLint)

11.2 Test Coverage

Test File	Cases	Coverage
eonet-normalization.test.ts	34	EONET category mapping, geometry selection, coordinate validation, URL sanitization
hotspot-score.test.ts	15	Hotspot score boundaries, determinism, all scoring factors
personal-relevance.test.ts	16	Personal relevance boundaries, weight computation
distance.test.ts	13	Haversine distance, coordinate validation, edge cases
health.test.ts	N/A	API health check endpoint
event-visuals.test.ts	N/A	Visualization utility functions

11.3 Testing Principles

- Scoring algorithm tests must cover boundary conditions (score of 0, 100, extreme inputs)
- All data normalization tests cover both valid and invalid inputs
- Distance tests cover identical coordinates, equator, poles, and date-line scenarios
- Tests use a fixed referenceDate to ensure determinism

12 Privacy & Data Ethics

12.1 Privacy Design

- No IP addresses are stored or logged
- No location information logged anywhere
- Location data is derived server-side from Vercel Headers only (city-level precision)
- No location-tracking cookies are set
- Precise browser geolocation is strictly opt-in (requires explicit button click)
- No location data is persisted to any database or storage
- No user account system
- No authentication mechanism
- No tracking or analytics code

12.2 Data Source Disclaimer

Planet Pulse uses satellite and public environmental data for situational awareness and educational exploration. It explicitly declares: this is NOT an official emergency alert service. Do not rely on it for safety-critical decisions.

All data originates from publicly available NASA APIs. No user uploads or user-generated content are involved.

12.3 Transparent Data Limitations

- EONET events are sourced from multiple international agencies; not all events are captured in real time
- Some events lack magnitude or scientific measurement data
- The hotspot score is a relative ranking tool, not a scientific hazard assessment
- FIRMS detects thermal anomalies (hot pixels) which may include non-wildfire heat sources
- IP geolocation precision is city level at best
- EONET API may be unavailable; degraded data is clearly labeled

13 Future Roadmap (Phase 2)

The following features are planned for Phase 2, ordered by priority:

Feature	Description
CesiumJS Integration	Replace react-globe.gl for higher-fidelity Earth rendering
NASA GIBS Layers	Satellite imagery layer overlay (temperature, vegetation, aerosols, etc.)
Event History Timeline	Timeline playback with scrub controls
Air Quality Data	Integration of AQI and other air quality datasets
Event Clustering	Aggregated markers at distant zoom levels
Shareable URLs	Event deep linking (URL params partially implemented)
PWA Support	Offline globe viewing
Multi-language	Internationalization support

13.1 Architectural Evolution

- Globe component boundary is already reserved; CesiumJS replacement only requires swapping the implementation layer
- Scoring system is fully modular; adding new data sources only requires a new API Route + normalization function
- Stateless design enables easy horizontal scaling

Thank you!

Planet Pulse - See what Earth is signaling now.

Copyright (c) 2026 John Zhou. Apache License 2.0.